Project Report

Parallel Kinematics of a Delta Robot

MA-MECH-25-VZ

Robotics

Author(s)	Andreas Thöni
Student ID(s)	52216067
Cohort	MA-MECH-25-VZ
Lecturer	Mehmet Ismet Can Dede, PhD

Contents

Chapter 1

Task description

The task of this project was to generate a model for the Delta 360 parallel manipulator by IGUS in MATLAB using the Simscape Multibody blockset. After generating the model, several MATLAB functions should be implemented to compare theoretical calculations of kinematics with the simulation, move in joint- and task-space, as well as create suitable trajectories.

Chapter 2

Simscape Multibody Modeling

The Multibody model of the robot has been created using the Simscape Multibody Link Plugin and several files that have been given by the lecturer. Using the plugin, it is possible to create a simulation from an exported .xml file and several .step-files, both exported from CAD-software, in this case from Solidworks. The resulting model has then been slightly adjusted to be usable for the tasks at hand. These adjustments include:

- the addition of a transform sensor, which is able to measure the relative transform between the origin of our coordinate system and the end-effector.
- the addition of several transform adjustments, to ensure the correct detection of the relative transform.
- the addition of position input ports to the prismatic joints, to be able to control them accordingly.
- the addition of three subsystems, which automatically compute the first two derivative of the input position information as well as perform a sign-correction.

A screenshot of how the adjusted model looks can be seen in figure 2.1.

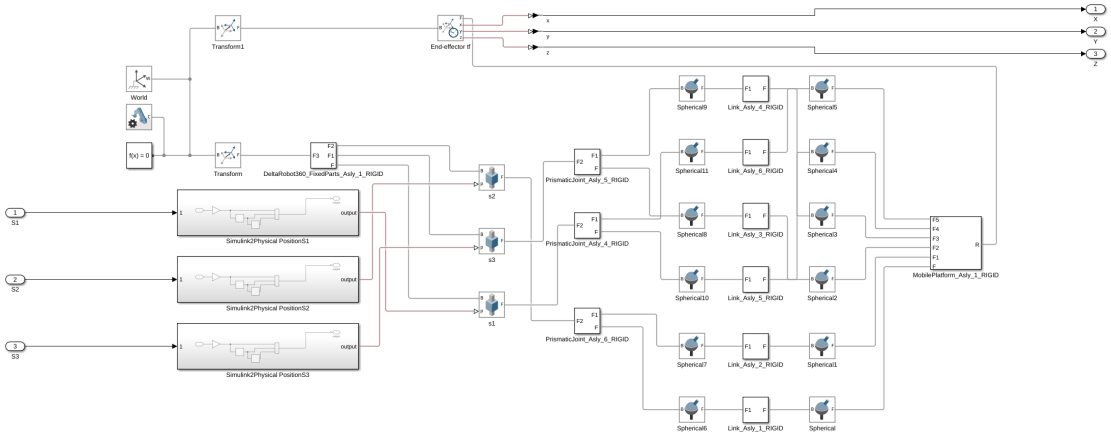

Figure 2.1: Adjusted Simscape Multibody model of the robot

Chapter 3

Forward and Inverse Kinematics

3.1 Forward Kinematics

Calculating forward kinematics is the process of determining the endeffector position depending on the joint variables. The equations for calculating the forward kinematics as well as the needed dimensions of the Delta robot have been taken from [1] and were put inside a MATLAB-function block to be usable inside Simulink. The code of the function block can be found in *forward_kinematics.m*.

To be able to verify if the equations have been implemented correctly, the euclidean error between the Multibody simulation (MB) and the output of the forward kinematics function (FK) has been calculated. The results can be seen in figure 3.1.

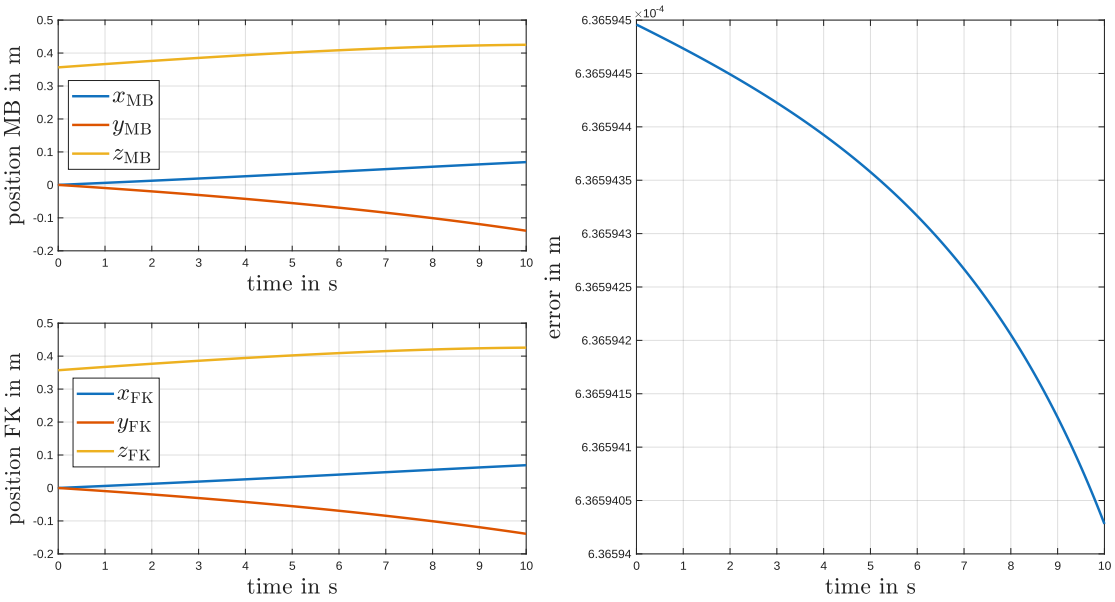

Figure 3.1: Plot of Verification process for forward kinematics. Left column: position calculated by Multibody model (MB) and function (FK); right column: euclidean error of the two

The left column shows that for the joint trajectory that has been given to the system, the position

of the end-effector in simulation and the forward kinematics function yield the same result. The plot in the right column shows an error of magnitudes of $10^{-4}\,\mathrm{m}$.

3.2 Inverse Kinematics

Calculating inverse kinematics is the process of determining the joint positions necessary for the end-effector to be at a given position. The equations for inverse kinematics for the robot are also taken from [1]. A function called *inverse_kinematics.m* has been implemented to do the conversion. To verify the results, the end-effector output of the Multibody-model has been fed into the inverse kinematics function, whose output has been compared to the input into the Simulation. Like above, the inputs are three ramps with different inclinations. The results can be seen in figure 3.2.

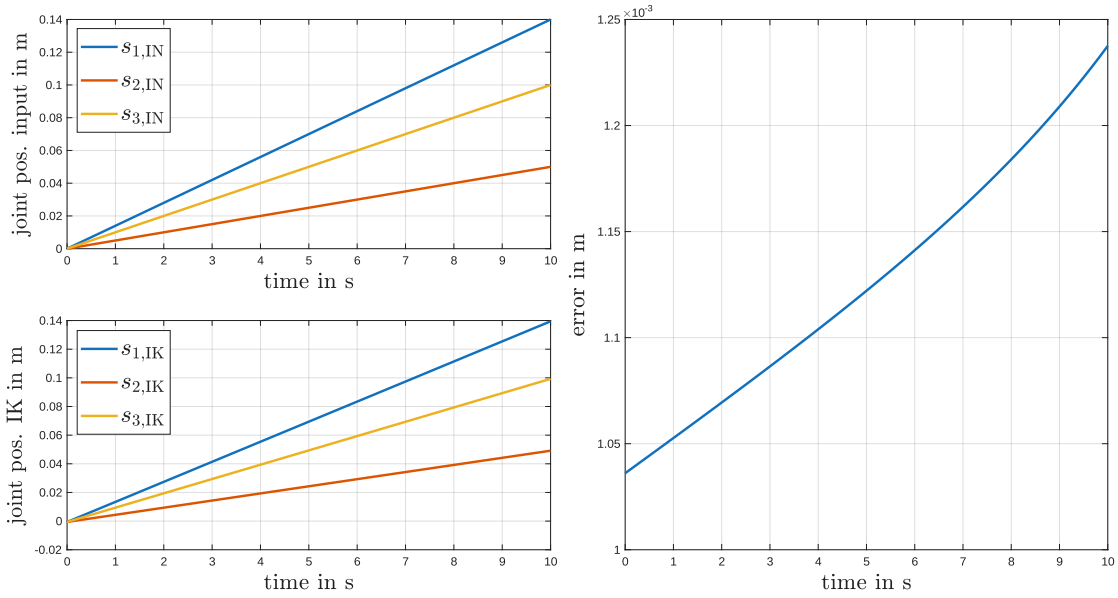

Figure 3.2: Plot of Verification process for inverse kinematics. Left column: input joint position into Multibody model (IN) and output of inverse kinematics function (IK); right column: euclidean error of the two

It is apparent, that the inverse kinematics (IK) function yields the same results as the joint positions that were input into the simulation. The error, however, has a magnitude of $10^{-3}\,\mathrm{m}$, which is larger than for forward kinematics.

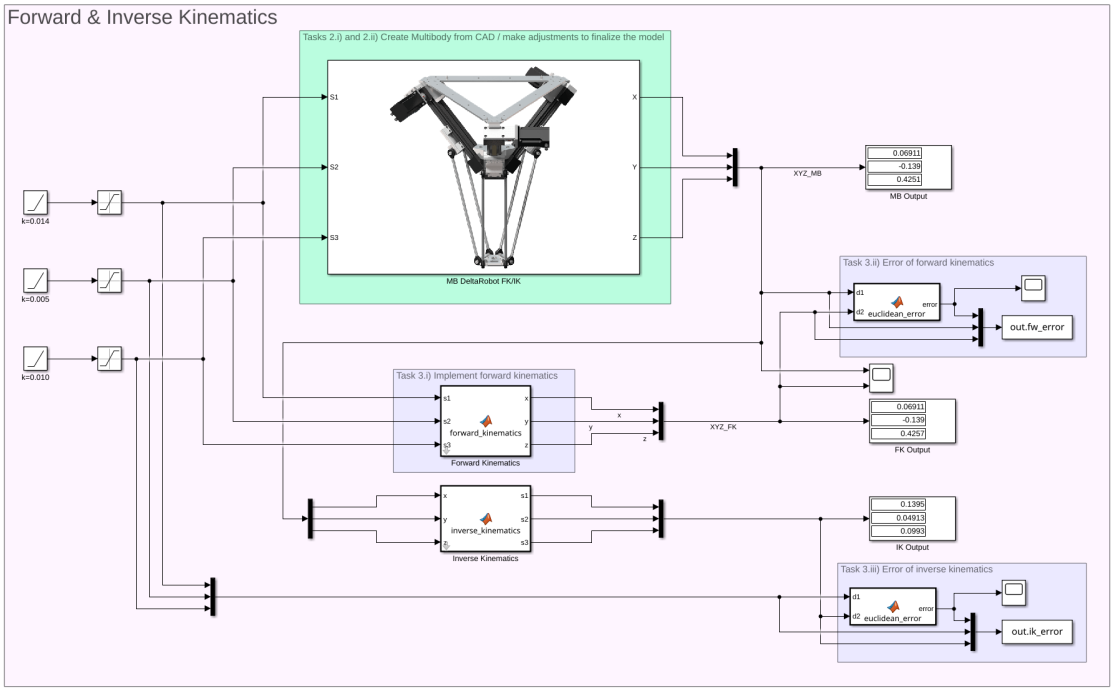

Figure 3.3: Simulink layout for verifying forward and inverse kinematics

A screenshot of the section of the Simulink model that has been used to fulfill these tasks can be seen in figure 3.3.

Chapter 4

Joint Space Motion

In this section, the process of oding joint space motions with the robot is documented. First, three waypoints are chosen inside of the workspace of the robot. They can be seen in table 4.1.

Table 4.1: Chosen waypoints for trajectory planning

Waypoint	x	y	z
1	0.0	0.0	0.4
2	0.1	0.1	0.4
3	−0.1	−0.1	0.4
4	0.1	−0.1	0.4
5	0.0	0.0	0.0

Using the acceleration and velocity constraints that were given in the task, the synchronized velocity profile has been calculated in the file *jspace_trapz.m*. The outputs are time stamps and corresponding velocity values, which are then input into a Repeating table block in Simulink and fed into an integrator to receive the needed position information. As the simulation uses the vectors from the workspace, the script *jspace_trapz.m* needs to be run before the simulation.

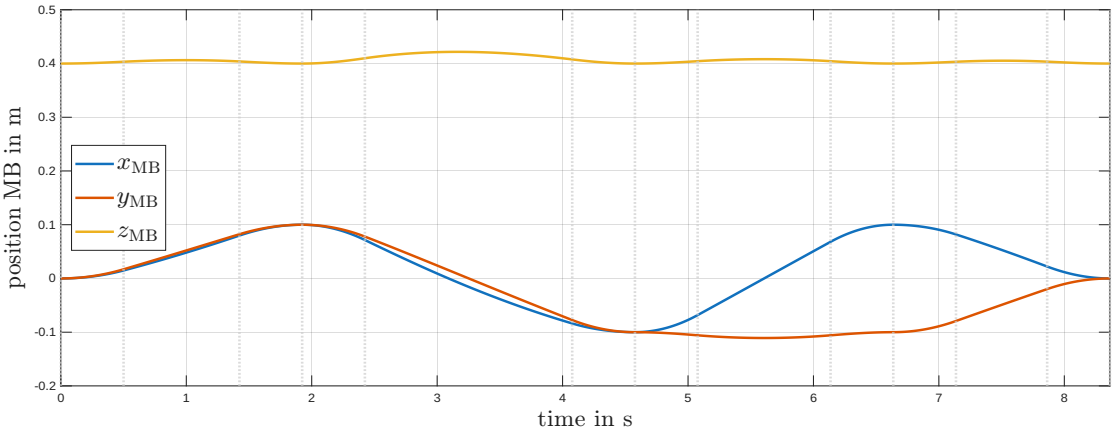

Figure 4.1: x, y and z position of the end-effector for the trapezoidal velocity profile

Figure 4.1 shows the position of the end-effector of the robot when following the trapezoidal trajectory. The trajectory itself, with position, velocity and acceleration profile can be viewed in figure 4.2.

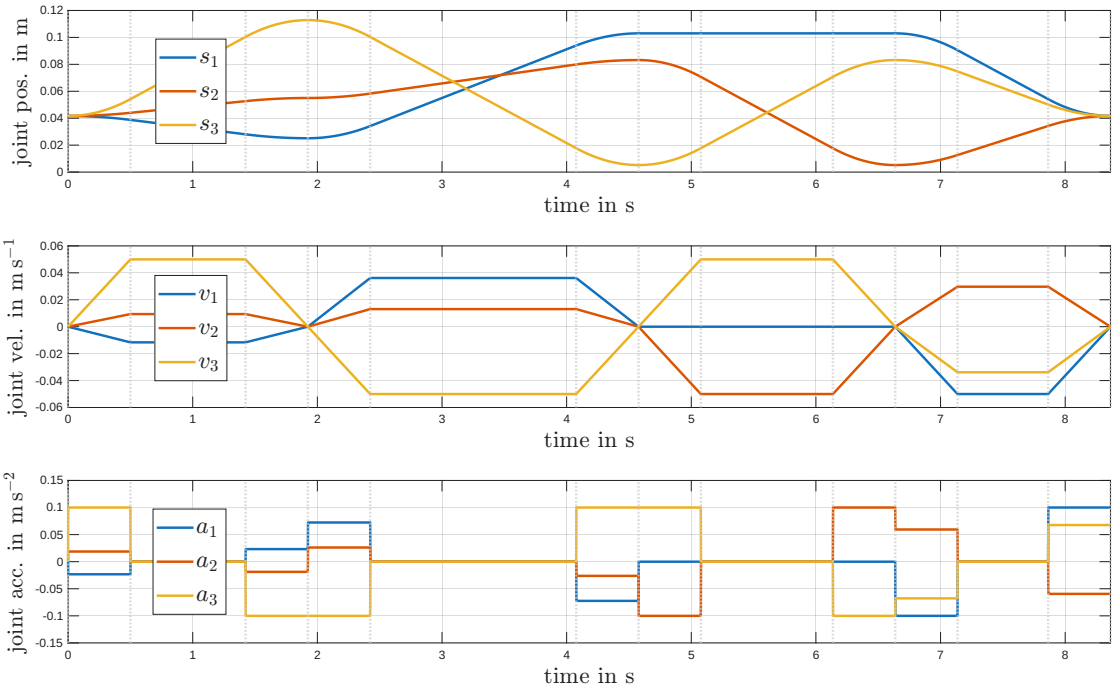

Figure 4.2: Generated trajectory, including position, velocity and acceleration profiles

Chapter 5

Question 4: Laurent Expansion

Chapter 6

Question 5: Evaluating the complex integral

Bibliography

[1] Prof. Dr. Mehmet Ismet Can Dede, *Parallel Manipulators: Kinematic Analyses of Delta 360 by IGUS*, MCI, 2021.